

Trabajo Práctico Integrador

Alumnos:

Mateo Peralta.

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Organización Empresarial

Docente Titular

Gabriela Martínez

Docente Tutor

Roberto Fabian Regner

18 de Junio de 2026

CASO QUE SE ABORDA.....	3
ENFOQUE SISTÉMICO.....	3
DIAGRAMAS.....	5
MÁQUINA DE ESTADOS DEL CHATBOT.....	5
MATRIZ DE TRANSICIONES.....	7
DICCIONARIO DE DATOS.....	8
MANUAL DE USUARIO.....	9
TEST DEL BOT.....	10
INTERACCIÓN CON INTELIGENCIA ARTIFICIAL.....	11
CONCLUSIÓN.....	13
ENLACES.....	13
BIBLIOGRAFÍA.....	15

CASO QUE SE ABORDA.

Solicitud de vacaciones:

El proyecto aborda la problemática de la carga administrativa en el departamento de Recursos Humanos de una organización en relación con la gestión, control y solicitud de licencias por vacaciones del personal.

Tradicionalmente, este proceso implica canales informales como mails, mensajes, planillas manuales, lo que genera demoras, pérdida de datos y falta de transparencia para el empleado respecto a su saldo disponible.

Para resolverlo, se diseñó e implementó un sistema de autogestión de RRHH mediante un bot de Telegram conectado a una base de datos relacional. El sistema automatiza la consulta de días disponibles y procesa las solicitudes de vacaciones de manera autónoma aplicando reglas de negocio predefinidas, aprobación automática si el empleado cuenta con saldo suficiente, optimizando los tiempos de respuesta y liberando tiempo operativo en la organización.

ENFOQUE SISTÉMICO.

Para comprender la dimensión e impacto de la solución de software desarrollada, es necesario analizar la organización y el bot bajo los fundamentos de la Teoría General de Sistemas. En el ámbito de la programación y el diseño de soluciones tecnológicas, incorporar esta perspectiva integral evita estudiar los componentes de manera aislada y permite visualizar cómo interactúan las partes para cumplir el propósito global del sistema.

Clasificación del Sistema:

Siguiendo los criterios de clasificación de la TGS expuestos en la cátedra, la organización donde se implementa el bot se define bajo los siguientes parámetros:

Según su interacción con el ambiente:

Es un sistema abierto. La empresa interactúa concurrentemente con su entorno intercambiando materia, energía e información (legislación laboral, dinámicas de mercado y flujos comunicacionales con los empleados).

Según su origen:

Es un sistema artificial, dado que constituye una estructura social y tecnológica creada deliberadamente por el hombre para cumplir objetivos específicos.

Según su complejidad:

Se categoriza como un sistema complejo, debido a la multiplicidad de variables dinámicas e interacciones que componen sus subsistemas corporativos.

Parámetros del Sistema:

A través del modelado abstracto, se definen los límites que separan el sistema de su ambiente y se identifican las variables críticas que gobiernan el proceso de transformación:

Entradas:

Insumos que el sistema recibe del entorno externo. Incluyen los identificadores únicos de comunicación id_telegram, los comandos transaccionales del usuario /start, /solicitar, días requeridos y el marco regulatorio externo que rige las licencias.

Procesos:

Operaciones internas ejecutadas asincrónicamente por el script de Python. La lógica embebida evalúa algebraicamente las variables mediante una Máquina de Estados Finitos y valida las reglas de negocio contrastándolas con la base de datos relacional.

Salidas:

Resultados entregados al entorno. Se materializan en las respuestas proyectadas en la interfaz de Telegram del empleado, la actualización del estado contable del saldo de vacaciones y el envío automatizado de alertas de auditoría al subsistema de supervisión en el canal correspondiente.

Retroalimentación:

Mecanismo de control interno. Una vez emitida una salida, el saldo modificado en la base de datos reconfigura de forma inmediata los parámetros de control para la siguiente consulta, ajustando y optimizando el funcionamiento del sistema en tiempo real.

Aplicación de Propiedades Sistémicas de la Organización:

Globalismo o Totalidad:

El comportamiento organizacional no puede entenderse analizando sus partes aisladas. El bot integra tecnológicamente las decisiones locales del departamento de recursos humanos con el rendimiento global de la organización, asegurando que la información fluya de manera correcta y eficiente.

Interdependencia:

Los subsistemas están interrelacionados y un cambio en uno afecta al todo. La automatización de solicitudes impacta directamente en la sincronía operativa entre las áreas de producción y la administración de personal.

Homeostasis y Adaptabilidad:

Representa la capacidad del sistema para mantener su equilibrio interno y responder a los cambios del contexto. Ante variaciones externas del entorno los canales automatizados garantizan estabilidad operativa mediante el monitoreo de indicadores de saldos.

Jerarquía de Sistemas:

La empresa opera con múltiples niveles jerárquicos dentro de un suprasistema mayor. El bot respeta este orden al descentralizar las decisiones de carácter operativo y automatizable en la base, mientras escala alertas de auditoría hacia los niveles superiores de supervisión.

Equipfinalidad:

Principio por el cual un sistema abierto puede alcanzar el mismo estado final o meta a partir de condiciones iniciales diferentes y por distintos caminos. El sistema logra el objetivo de control de vacaciones de forma flexible, adaptándose a los recursos tecnológicos disponibles y optimizando los tiempos del personal.

DIAGRAMAS.

Diagrama BPMN AS-IS:

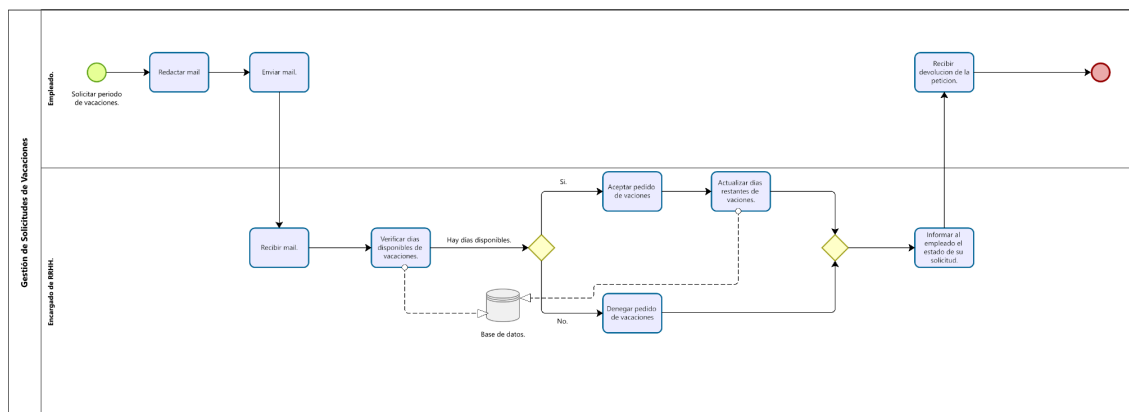
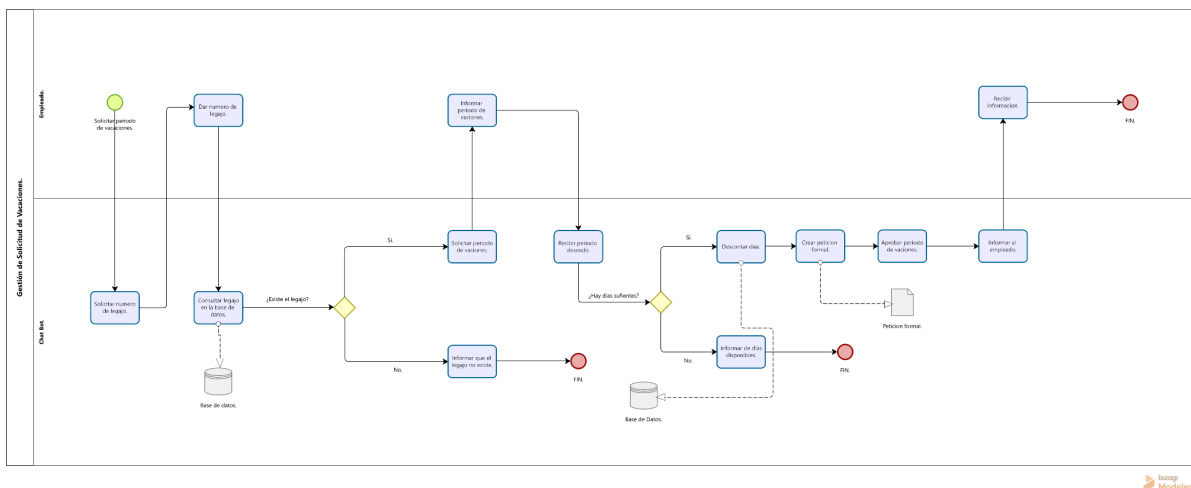


Diagrama BPMN TO-BE:



MÁQUINA DE ESTADOS DEL CHATBOT.

Siguiendo el modelo de Máquina de Estados, el chatbot puede encontrarse en un único estado a la vez y cambiar de estado según los eventos generados por el usuario.

Definición de Componentes:

Estados.

INICIO.

Estado inicial del chatbot. Espera que el usuario inicie una solicitud.

SOLICITANDO_LEGAJO.

El sistema solicita al usuario que ingrese su número de legajo.

VALIDANDO_LEGAJO.

El chatbot consulta la base de datos para verificar la existencia del empleado.

SOLICITANDO_DIAS.

El sistema solicita la cantidad de días de vacaciones requeridos.

VALIDANDO_DISPONIBILIDAD.

Se verifica si el empleado posee días suficientes para realizar la solicitud.

SOLICITUD_APROBADA.

El sistema registra la solicitud y actualiza la base de datos.

SOLICITUD_RECHAZADA.

El sistema informa que no es posible realizar la solicitud.

FIN.

Finalización del proceso.

Eventos / Entradas.

- INICIAR_SOLICITUD: El usuario selecciona la opción de solicitar vacaciones.
- LEGAJO_INGRESADO: El usuario ingresa un número de legajo.
- LEGAJO_VALIDO: El legajo existe en la base de datos.
- LEGAJO_INVALIDO: El legajo no existe.
- DIAS_INGRESADOS: El usuario indica la cantidad de días solicitados.
- DIAS_SUFICIENTES: El empleado posee saldo disponible.
- DIAS_INSUFICIENTES: El empleado no posee saldo suficiente.
- CANCELAR: El usuario cancela la operación.

- REGISTRO_COMPLETADO: La solicitud fue almacenada correctamente.

Acciones.

- Solicitar legajo.
- Consultar base de datos.
- Solicitar cantidad de días.
- Verificar disponibilidad.
- Registrar solicitud.
- Actualizar saldo de vacaciones.
- Informar aprobación.
- Informar rechazo.
- Finalizar proceso.

MATRIZ DE TRANSICIONES.

Estado Actual.	Evento.	Estado Siguiente	Acción.
INICIO.	INICIAR_SOLICITUD.	SOLICITANDO_LEGajo.	Solicitar legajo.
SOLICITANDO_LEGajo.	LEGAJO_INGRESADO.	VALIDANDO_LEGajo.	Consultar base de datos.
VALIDANDO_LEGajo.	LEGAJO_VALIDO.	SOLICITANDO_DIAS.	Solicitar cantidad de días.
VALIDANDO_LEGajo.	LEGAJO_INVALIDO.	SOLICITANDO_LEGajo.	Mostrar error y solicitar nuevamente.
SOLICITANDO_DIAS.	DIAS_INGRESADOS.	VALIDANDO_DISPONIBILIDAD.	Verificar saldo disponible.
VALIDANDO_DISPONIBILIDAD.	DIAS_SUFICIENTES.	SOLICITUD_APROBADA.	Registrar solicitud y actualizar saldo.
VALIDANDO_DISPONIBILIDAD.	DIAS_INSUFICIENTES.	SOLICITUD_RECHAZADA.	Informar rechazo.

SOLICITUD_APROBADA.	REGISTRO_COMPLETADO.	FIN.	Informar aprobación.
SOLICITUD_RECHAZADA.	CANCELAR.	FIN.	Finalizar proceso.

DICCIONARIO DE DATOS.

Entidades y Variables:

Descripción de Entidades Base de Datos - SQLite.

Las entidades representan las tablas relacionales diseñadas para persistir la información del sistema en el archivo recursos_humanos.db.

Entidad: empleados

Representa a los usuarios del sistema, personal de la empresa habilitados para utilizar la plataforma de autogestión.

id_telegram (Clave Primaria / INTEGER): Identificador único numérico asignado por la API global de Telegram a cada cuenta. Vincula directamente el chat físico con el registro de la base de datos.

nombre (TEXT): Nombre y apellido completo del empleado.

dias_disponibles (INTEGER): El saldo neto de días de vacaciones que el trabajador tiene a favor. Se decrementa automáticamente tras una aprobación exitosa.

Entidad: estados_bot

Entidad técnica utilizada para registrar la persistencia de la Máquina de Estados Finitos (FSM). Evita la pérdida de contexto ante desconexiones.

id_telegram (Clave Primaria / INTEGER): ID único del empleado en interacción activa.

estado_actual (TEXT): Almacena la situación del flujo en la que quedó congelada la conversación del usuario. Valores posibles: ['INICIO', 'ESPERANDO_DIAS'].

Entidad: solicitudes

Registro histórico y de auditoría de cada requerimiento de licencia cargado en el sistema.

id_solicitud (Clave Primaria Autoincremental / INTEGER): Código numérico secuencial generado de forma automática por el motor para identificar unívocamente cada pedido.

id_telegram (Clave Foránea / INTEGER): Referencia al empleado solicitante (relacionado con empleados.id_telegram).

cantidad_dias (INTEGER): Volumen de días corridos de vacaciones solicitados en el evento actual.

estado (TEXT): Condición final del trámite. Valores posibles: ['APROBADO', 'RECHAZADO'].

Descripción de Variables Manejadas. Lógica de Programación - Python.

Representan las variables volátiles utilizadas en la memoria RAM del script chat.py durante el procesamiento asincrónico del flujo de control:

TOKEN (String / Constante): Almacena la clave de autenticación e interfaz secreta provista por BotFather necesaria para que nuestro script se conecte y controle los servidores de la API de Telegram.

ID_JEFE (Integer / Constante): Almacena el número de ID de Telegram del supervisor o canal de RRHH. Se utiliza como dirección de destino obligatoria para el envío de alertas de auditoría automática.

id_usuario / id_empleado (Integer): Variable dinámica que captura el ID del remitente desde el objeto update.message.chat_id para realizar las consultas de saldo y estado específicas de esa persona.

estado (String): Variable local que recupera de la base de datos la situación actual del usuario para evaluar qué bloque lógico debe disparar el despachador de mensajes.

texto_recibido (String): Almacena el mensaje de texto plano que el usuario acaba de tipear en el chat, utilizado para evaluar si introdujo comandos o si ingresó el número de días.

dias_pedidos (Integer): Conversión numérica (Casteo de String a Int) del texto enviado por el empleado cuando el bot se encuentra en el estado ESPERANDO_DIAS. Se usa para realizar la comparación matemática contra el saldo.

saldo_disponible (Integer): Almacena temporalmente el valor numérico recuperado de la columna dias_disponibles de la base de datos para validar físicamente la viabilidad de la regla de negocio.

nombre / nombre_emp (String): Almacena el string del nombre del empleado recuperado de la base de datos, utilizado para personalizar las respuestas del bot y armar la alerta que le llega al supervisor.

id_sol (Integer): Variable que captura el número de ID autoincremental de la fila recién insertada en la tabla solicitudes, necesario para imprimir el código de trámite (ej: #1) en los mensajes de confirmación.

MANUAL DE USUARIO.

Para el Empleado:

Inicio del Bot: El usuario debe buscar el bot en Telegram por su nombre de usuario Chat_Vacaciones_bot y presionar el botón Iniciar o enviar el comando /start. El bot lo saludará por su nombre y le indicará cuántos días de vacaciones tiene disponibles.

Iniciar una Solicitud: Para pedir días, se debe enviar el comando /solicitar. El bot cambiará de estado y le pedirá que introduzca la cantidad de días deseados.

Ingreso de Días: El usuario debe escribir un número entero.

Si posee saldo suficiente, el bot responderá inmediatamente confirmando la Aprobación Automática.

Si pide más días de los disponibles, el sistema notificará que la solicitud fue denegada.

TEST DEL BOT.

Caso de Prueba 1:

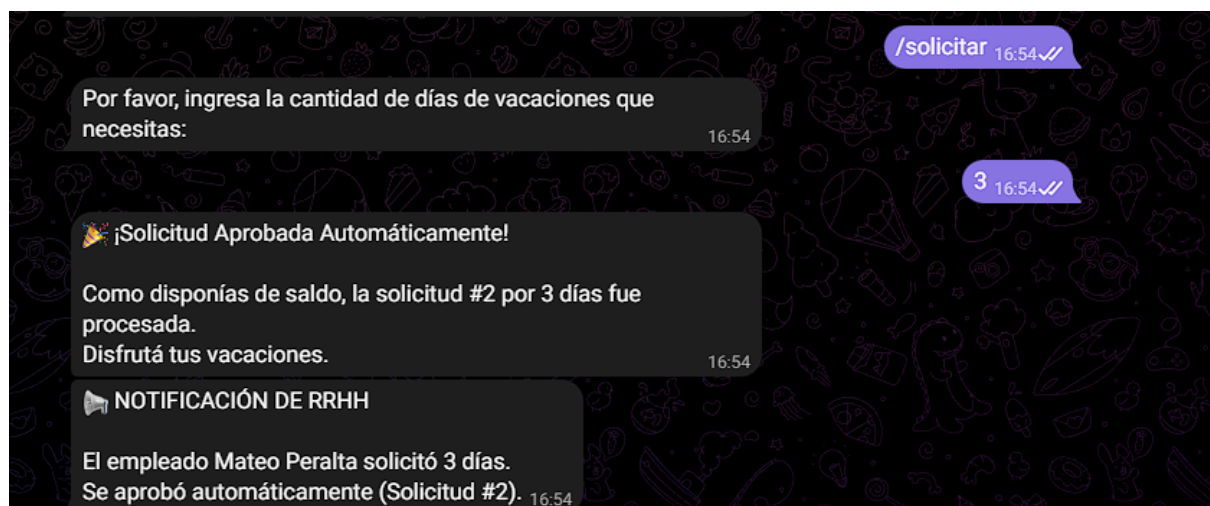
Flujo de Solicitud Exitosa.

Estado Inicial: Usuario registrado con 14 días disponibles. Estado: INICIO.

Acción: El usuario ingresa y envía el número 3.

Resultado Esperado: El bot confirma la aprobación automática de la solicitud #2.

Al ejecutar /start nuevamente, el saldo actualizado del empleado debe ser de 11 días. El supervisor recibe la notificación del pedido.



Caso de Prueba 2:

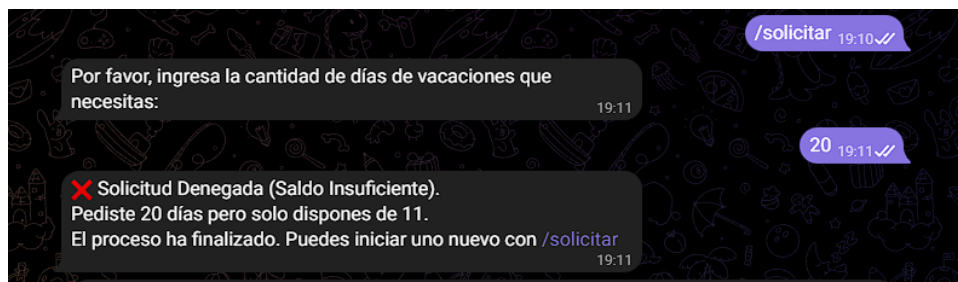
Rechazo por Saldo Insuficiente Estado Inicial:

Usuario con 11 días disponibles.

Estado: INICIO.

Acción: El usuario ingresa y el número 20.

Resultado Esperado: El bot detecta que $20 > 11$, emite el mensaje "Solicitud Denegada (Saldo Insuficiente)" y regresa el estado a INICIO sin alterar los registros de saldo.



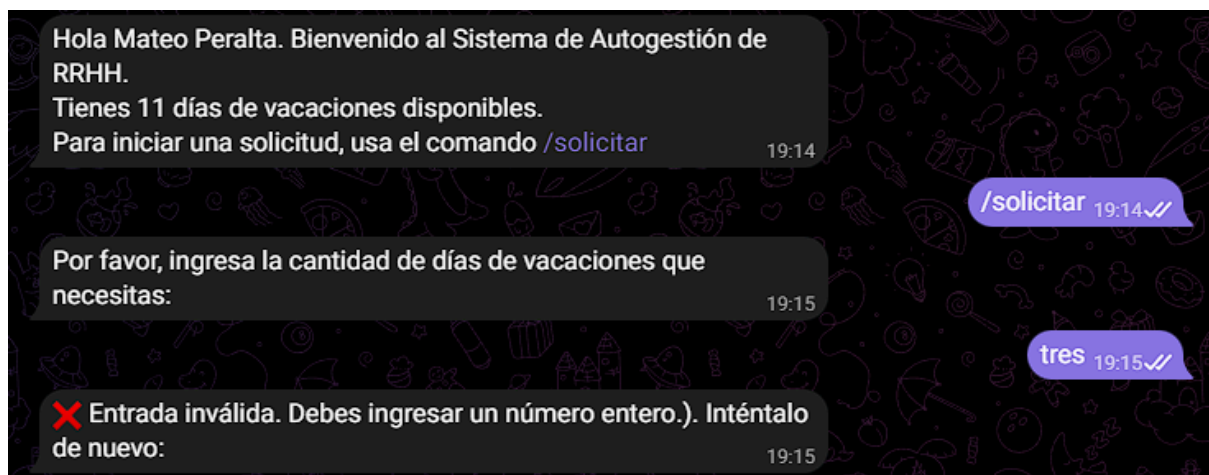
Caso de Prueba 3:

Entrada Inválida manejador de Errores

Estado Inicial: Estado de la conversación: ESPERANDO_DIAS.

Acción: El usuario envía el texto "tres" en lugar del número entero 3.

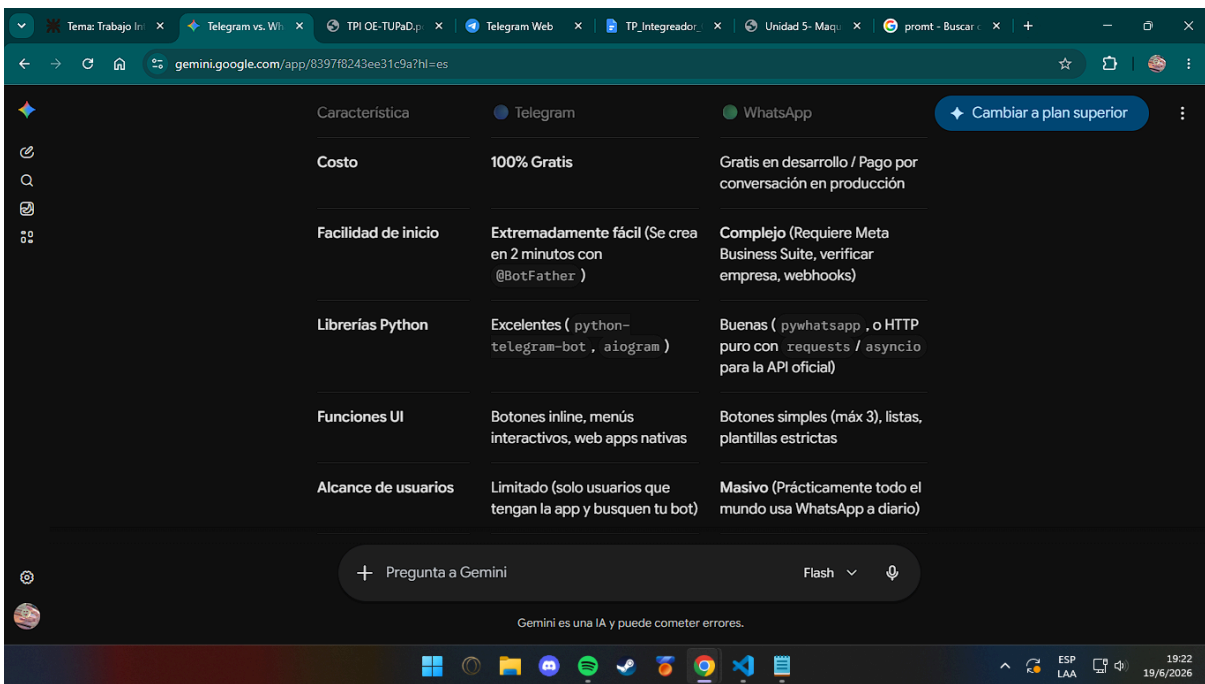
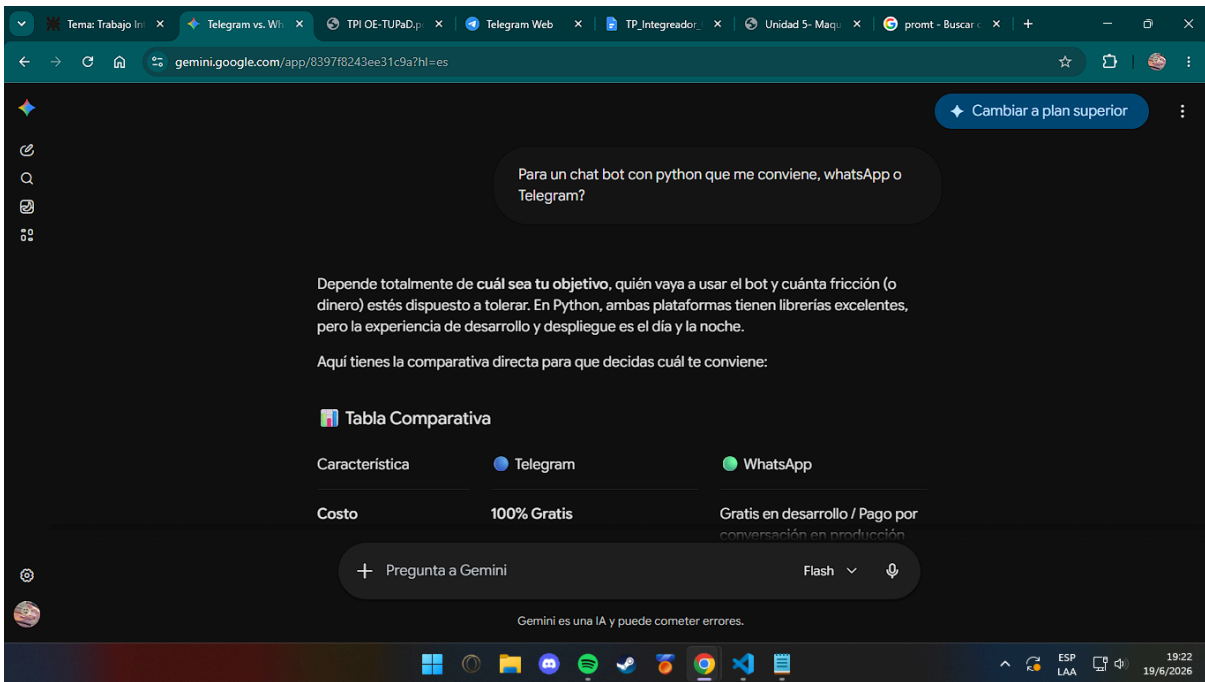
Resultado Esperado: El bot identifica que la entrada no es un dígito, envía un mensaje de advertencia "Entrada inválida. Debes ingresar un número entero" y mantiene el estado en ESPERANDO_DIAS hasta recibir una entrada correcta.



INTERACCIÓN CON INTELIGENCIA ARTIFICIAL.

Durante el ciclo de desarrollo del proyecto, se utilizó el modelo de IA Gemini como herramienta de asistencia técnica en ingeniería de software. Las interacciones se centraron en dos hitos principales: en primer lugar, el asesoramiento para la configuración del entorno de ejecución, permitiendo adaptar y

resolver incompatibilidades críticas de librerías en la versión moderna de Python 3.14. En segundo lugar, se empleó para la depuración del código fuente y la corrección de errores de importación asincrónica, optimizando los tiempos de desarrollo y garantizando la robustez del sistema final.



```
PS C:\Users\mateo\Desktop\Codigo_TP_OE> &
C:/Users/mateo/AppData/Local/Programs/Python/Python314/
python.exe c:/Users/mateo/Desktop/Codigo_TP_OE/chat.py
C:\Users\mateo\AppData\Local\Programs\Python\Python314>
```

¡Qué pesadilla! Python 3.14 rompió por completo la estructura interna de esa versión de Telegram y cada parche nos lleva a otro error de importación profunda (`urllib3.contrib.appengine` ya no existe en las versiones modernas de `urllib3` que instalamos).

Como tenés que terminar hoy, andar emparchando una librería vieja contra un Python del futuro te va a consumir las horas que te quedan. Vamos a hacer el cambio más inteligente y rápido: pasar a la versión moderna y mantenida de la librería (`python-telegram-bot` versión 20+), que es nativamente compatible con Python 3.14 y no necesita ningún parche raro.

Como tenés que terminar hoy, andar emparchando una librería vieja contra un Python del futuro te va a consumir las horas que te quedan. Vamos a hacer el cambio más inteligente y rápido: pasar a la versión moderna y mantenida de la librería (`python-telegram-bot` versión 20+), que es nativamente compatible con Python 3.14 y no necesita ningún parche raro.

✦ Cambiar a plan superior

Solo tenemos que desinstalar lo viejo, instalar lo nuevo y adaptar el código (que queda mucho más limpio).

Paso 1: Limpiar e Instalar lo correcto

Corré estos dos comandos en tu terminal:

Bash

```
pip uninstall python-telegram-bot urllib3 six filetype -y
pip install python-telegram-bot --upgrade
```

CONCLUSIÓN.

La implementación del Bot de Autogestión de RRHH demuestra el impacto positivo que tiene la automatización de procesos sobre la eficiencia organizacional. Al delegar la verificación de saldos y la aprobación de solicitudes en un sistema determinista guiado por una Máquina de Estados, se eliminaron los cuellos de botella y las demoras humanas asociadas a las autorizaciones tradicionales.

Técnicamente, el uso de programación asincrónica en Python junto con la persistencia en una base de datos relacional como SQLite demostró ser una arquitectura robusta, económica y escalable. El sistema no solo garantiza la integridad de los datos frente a fallos imprevistos, sino que también establece un canal transparente y ágil de comunicación interna, cumpliendo con los estándares modernos de transformación digital requeridos por las organizaciones actuales.

ENLACES.

[Repositorio de Github:](#)

https://github.com/Mateo-140/TP_Integrador_OE

Contenido:

Archivos .py (chat.py y base_datos.py): Código fuente que contiene la lógica asincrónica del bot de Telegram, el control de la máquina de estados y las consultas SQL.

Base de Datos (recursos_humanos.db): Archivo que almacena de manera persistente las tablas relacionales de empleados, estados_bot y solicitudes.

Modelos de Bizagi: Diagramas y archivos de modelado BPMN que documentan el rediseño digital del proceso de solicitudes de Recursos Humanos.

README.md: Documentación técnica en formato Markdown con la guía de instalación, configuración del Token, ejecución y los casos de prueba del sistema.

.gitignore: Archivo de configuración que evita subir archivos basura del entorno virtual, configuraciones locales o credenciales confidenciales al servidor de GitHub.

Bot de Telegram:

t.me/Chat_Vacaciones_bot

BIBLIOGRAFÍA.

Python Software Foundation. (2026). Python Language Reference, versión 3.14. Disponible en <https://docs.python.org>

Python-telegram-bot Developers. (2023). Python Telegram Bot Documentation (v20.x). Disponible en <https://python-telegram-bot.org/>

Hipp, D. R., et al. (2026). SQLite Database Engine y sentencias SQL. Disponible en <https://www.sqlite.org>

Cátedra de Organización Empresarial. (2025). Guía de Trabajos Prácticos e Integradores: Modelado de Procesos y Máquinas de Estado. Tecnicatura Universitaria en Programación, UTN.